

Universitatea Politehnica Timișoara
Facultatea de Automatică și Calculatoare
Ingineria Sistemelor
Anul universitar 2025–2026

**SISTEM SMART HOME LOCAL-FIRST CU
DETECTIE DE ANOMALII
COMPORTAMENTALE BAZATĂ PE PROCESARE
EDGE**

Candidat: Vidac Denisa

Coordonator științific: Conf./Prof. dr. ing. _____

Sesiunea: Iunie 2026

REZUMAT

Lucrarea de față prezintă proiectarea și implementarea unui sistem de automatizare rezidențială (*smart home*) local-first, care funcționează în mod complet independent față de orice infrastructură cloud externă. Sistemul este construit pe o arhitectură în cinci straturi, utilizând microcontrolere ESP32 WROOM-32 pentru colectarea datelor de la senzori (temperatură, umiditate, gaz, mișcare, lumină) și controlul actuatorilor (releu de putere, servomotor, buzzer), un modul ESP32-CAM pentru supravegherea video, un gateway Raspberry Pi 3B+ care găzduiește broker-ul MQTT Mosquitto, backend-ul NestJS, baza de date TimescaleDB și motorul de detecție a anomaliilor, și o aplicație mobilă React Native/Expo pentru interfața utilizatorului.

Contribuția principală a lucrării constă în integrarea unui mecanism de detecție a anomaliilor comportamentale bazat pe algoritmul Isolation Forest, rulat exclusiv pe Raspberry Pi, care identifică devierile de la tiparele normale fără a transmite datele personale în afara rețelei locale. Sistemul demonstrează că o soluție IoT de nivel competitiv poate fi construită integral cu hardware accesibil (sub 150 EUR) și tehnologii open-source, oferind o alternativă viabilă la ecosistemele comerciale Google Home, Amazon Alexa sau Tuya.

Cuvinte cheie: IoT, smart home, local-first, edge computing, ESP32, Raspberry Pi, MQTT, NestJS, React Native, Isolation Forest, detecție anomalii.

Cuprins

1	Introducere	5
1.1	Contextul și Motivația Proiectului	5
1.2	Obiectivele Lucrării	6
1.3	Contribuția Proprie	7
1.4	Structura Lucrării	8
2	Arhitectura Sistemului	9
2.1	Viziunea Arhitecturală	9
2.1.1	Principiul Local-First	9
2.1.2	Confidențialitatea prin Proiectare	9
2.1.3	Inteligența la Marginea Rețelei față de Inteligența în Cloud	10
2.2	Arhitectura pe Straturi	10
2.2.1	Stratul 1 — Senzori și Actuatori	11
2.2.2	Stratul 2 — Noduri ESP32	12
2.2.3	Stratul 3 — Gateway Raspberry Pi 3B+	12
2.2.4	Stratul 4 — Backend API NestJS	13
2.2.5	Stratul 5 — Aplicația Mobilă React Native/Expo	13
2.3	Fluxul de Date (End-to-End)	13
2.3.1	Fluxul de Monitorizare (Senzori → Mobil)	13
2.3.2	Fluxul de Comandă (Mobil → Actuator)	14
2.3.3	Fluxul de Alertare (Anomalie ML → Notificare)	14
2.4	Alegerea Tehnologiilor	14
2.4.1	NestJS față de alternativele backend	14
2.4.2	React Native față de Flutter	15
2.4.3	MQTT față de HTTP polling	15
2.4.4	TimescaleDB față de PostgreSQL simplu	15
2.4.5	Isolation Forest față de reguli cu praguri fixe	16
2.5	Considerații de Securitate	16
3	Implementarea Hardware	18
3.1	Prezentare Generală	18
3.2	Microcontrolere Utilizate	18
3.2.1	ESP32 WROOM-32 — Nodul de Senzori	18
3.2.2	ESP32-CAM AI-Thinker — Nodul Cameră	19
3.3	Senzori Utilizați	19
3.3.1	DHT22 — Temperatură și Umiditate	19

3.3.2	MQ-2 — Gaz și Fum	20
3.3.3	HC-SR501 — Mișcare PIR	20
3.3.4	BH1750 — Lumină Ambientală	20
3.4	Actuatori Utilizați	20
3.4.1	Modulul Releu 4 Canale	20
3.4.2	Buzzer Activ	21
3.4.3	Servomotorul SG90 — Control Perdele	21
3.5	Schema de Conexiuni	21
3.6	Alimentarea Sistemului	22
3.7	Nodul Cameră ESP32-CAM — Programare și Configurare	23
3.8	Considerații Practice pentru Breadboard	23
4	Implementarea Software	25
5	Detecția Anomaliilor prin Machine Learning	26
6	Testare și Evaluare	27
	Concluzii	28
	Bibliografie	29

Capitolul 1

Introducere

1.1 Contextul și Motivația Proiectului

Internetul Lucrurilor (IoT — *Internet of Things*) reprezintă una dintre cele mai semnificative transformări tehnologice ale ultimului deceniu. Conform rapoartelor Statista și IoT Analytics, numărul dispozitivelor IoT conectate la nivel global a depășit pragul de 15 miliarde în anul 2024, cu o prognoză de creștere spre 30 de miliarde până în 2030. În cadrul acestui ecosistem în expansiune rapidă, segmentul casei inteligente (*smart home*) ocupă o poziție de prim rang, atât din perspectiva adoptării consumatorilor, cât și din cea a volumului de investiții al marilor corporații tehnologice.

Piața globală a sistemelor smart home a fost evaluată la aproximativ 110 miliarde de dolari în 2024 și se estimează că va atinge 240 de miliarde de dolari până în 2030, cu o rată de creștere anuală compusă (CAGR) de aproximativ 13,5%. Această creștere este impulsivată de accesibilizarea dispozitivelor senzoriale, reducerea costurilor microcontrolerelor și creșterea penetrării rețelelor Wi-Fi în gospodăriile moderne. Soluțiile comerciale dominante — precum Google Home, Amazon Alexa și ecosistemul Tuya — au democratizat accesul la automatizarea casei, oferind interfețe intuitive și o gamă largă de dispozitive compatibile.

Cu toate acestea, analiza critică a acestor soluții comerciale relevă o serie de limitări structurale care ridică întrebări fundamentale legate de confidențialitate, fiabilitate și autonomie tehnologică. Marea majoritate a ecosistemelor smart home comerciale sunt construite în jurul unui model arhitectural *cloud-centric*, în care datele colectate de senzorii din locuință sunt transmise și stocate pe servere externe, gestionate de companii terțe. Astfel, un senzor de temperatură amplasat în dormitorul utilizatorului transmite permanent date către infrastructura cloud a producătorului, iar un dispozitiv PIR (*Passive Infrared*) pentru detectarea mișcării furnizează implicit informații detaliate despre comportamentul și prezența persoanelor în locuință.

Problemele asociate acestui model sunt multiple și de natură diversă. În primul rând, există o dependență critică de disponibilitatea internetului și a serviciilor cloud ale furnizorului. Numeroase cazuri documentate demonstrează că sistemele smart home bazate pe cloud devin complet nefuncționale în momentul în care furnizorul decide să întrerupă suportul pentru produse mai vechi sau când serverele de backend sunt temporar indisponibile. Cazul emblematic al platformei Wink, care a trecut brusc la un model prin abonament în 2020 lăsând utilizatorii fără acces la propriile dispozitive, sau decizia Google de a discontinua suportul pentru Nest Secure, ilustrează fragilitatea acestor dependențe. În al doilea rând, modelul cloud-centric implică transmiterea continuă a datelor personale sensibile — rutine comportamentale, orare de activitate, prezența sau absența persoanelor în locuință — către servere

externe, creând profile detaliate ale utilizatorilor cu implicații profunde pentru confidențialitate. Regulamentul General privind Protecția Datelor (GDPR) din Uniunea Europeană a adresat parțial aceste preocupări, însă mecanismele de aplicare rămân limitate în fața unor companii cu infrastructuri globale complexe.

În acest context, se identifică o oportunitate clară pentru proiectarea și implementarea unui sistem smart home alternativ, fundamentat pe principii arhitecturale diferite: localitate completă a datelor, procesare inteligentă la marginea rețelei (*edge computing*) și independență față de orice infrastructură cloud. Progresele recente în domeniul micro-controlerelor de putere redusă — în special familia ESP32 de la Espressif Systems — și disponibilitatea unor platforme de calcul la nivel de placă unică, precum Raspberry Pi, fac posibilă implementarea unui sistem complet funcțional la un cost de hardware semnificativ mai mic decât cel al soluțiilor comerciale echivalente.

1.2 Obiectivele Lucrării

Pornind de la analiza contextului și a problemelor identificate în secțiunea anterioară, prezenta lucrare urmărește realizarea unui set coerent de obiective, structurate pe mai multe planuri ale proiectării și implementării sistemului.

Primul obiectiv major constă în proiectarea și implementarea unui sistem smart home complet funcțional, bazat exclusiv pe infrastructură locală, fără nicio dependență de servicii externe sau conectivitate la internet pentru funcționarea de bază. Sistemul trebuie să asigure monitorizarea în timp real a parametrilor de mediu (temperatură, umiditate, concentrație de gaze, nivelul luminii, detectarea mișcării), controlul actuatorilor (releu de putere, motor servo, buzzer de alertă) și gestionarea unui nod de supraveghere video, toate acestea operând exclusiv în cadrul rețelei locale a utilizatorului.

Al doilea obiectiv vizează integrarea unui mecanism de detecție a anomaliilor comportamentale bazat pe tehnici de machine learning, rulat în întregime pe dispozitivul gateway (Raspberry Pi 3B+). Spre deosebire de sistemele clasice de alertă bazate pe praguri fixe (*threshold-based*), abordarea propusă utilizează algoritmul Isolation Forest [2] pentru a învăța tiparele normale de comportament ale utilizatorului și pentru a detecta deviațiile semnificative de la aceste tipare. Această capacitate conferă sistemului un grad de inteligență adaptivă, reducând rata de alerte fals-pozitive caracteristice sistemelor bazate pe reguli rigide.

Al treilea obiectiv se referă la realizarea unei aplicații mobile moderne, disponibile pe platformele iOS și Android, care să ofere utilizatorului o interfață intuitivă pentru monitorizarea stării sistemului, vizualizarea istoricului datelor, gestionarea alertelor și controlul de la distanță al actuatorilor. Aplicația trebuie să beneficieze de actualizări în timp real prin intermediul protocolului WebSocket, eliminând necesitatea interogării periodice (*polling*) a serverului și asigurând o experiență de utilizare fluidă și reactivă.

Al patrulea obiectiv constă în demonstrarea că un sistem IoT de nivel competitiv poate fi construit integral fără dependență de cloud, utilizând exclusiv tehnologii open-source

și hardware comercial accesibil. Această demonstrație are valoare atât din perspectiva practică, cât și din cea academică, contribuind la literatura de specialitate în domeniul arhitecturilor IoT locale și al procesării edge.

Al cincilea obiectiv, cu caracter pragmatic, stabilește un buget maxim de hardware de 150 EUR pentru întreaga infrastructură fizică a sistemului, incluzând microcontrolerele ESP32, senzorii, actuatorii, cablajul și placa Raspberry Pi. Respectarea acestei constrângeri de cost demonstrează accesibilitatea soluției pentru publicul larg și competitivitatea sa față de ecosistemele comerciale cu costuri lunare recurente.

1.3 Contribuția Proprie

Lucrarea de față reprezintă o contribuție originală pe mai multe planuri tehnice interconectate, autorul proiectând și implementând toate componentele sistemului de la zero, fără utilizarea de soluții preconstruite sau platforme *no-code*.

Firmware-ul pentru nodurile ESP32 a fost dezvoltat în MicroPython, oferind o soluție echilibrată între performanță și productivitate pentru gestionarea senzorilor și actualelor. Firmware-ul implementează conectivitatea MQTT cu autentificare și reconectare automată, rutine de citire pentru senzorii DHT22 (temperatură/umiditate), MQ-2 (gaze combustibile), PIR (mișcare) și BH1750 (luminanță), precum și controlul actualelor (releu, servo motor, buzzer).

Backend-ul NestJS constituie stratul de orchestrare al întregului sistem, expunând un API REST securizat cu autentificare JWT și un gateway WebSocket pentru transmiterea evenimentelor în timp real către aplicația mobilă. Arhitectura modulară a aplicației NestJS — cu module separate pentru autentificare, gestionarea senzorilor, controlul actualelor și alertare — permite extensibilitatea și mentenabilitatea codului. Integrarea cu TimescaleDB prin TypeORM și cu broker-ul MQTT prin biblioteca `mqtt.js` realizează bridging-ul între protocoalele de comunicație IoT și interfețele REST/WebSocket moderne.

Pipeline-ul de detecție a anomaliilor reprezintă componenta de machine learning a sistemului, implementată în Python pe Raspberry Pi 3B+. Algoritmul Isolation Forest, implementat prin biblioteca `scikit-learn`, este antrenat pe date istorice de comportament ale utilizatorului și rulat periodic pentru a evalua normalitatea citirilor curente ale senzorilor. Contribuția proprie include proiectarea schemei de antrenare incrementală, normalizarea datelor multidimensionale și integrarea rezultatelor algoritmului cu sistemul de alertare prin MQTT.

Aplicația mobilă React Native/Expo oferă o interfață utilizator modernă, cu o temă vizuală caldă bazată pe paleta de culori stone (gri-bej) și amber (chihlimbar), adaptată pentru utilizare nocturnă. Aplicația implementează autentificare JWT cu persistență locală, vizualizare în timp real a datelor senzorilor prin WebSocket, grafice de evoluție temporală și un sistem de notificări pentru alertele detectate de algoritmul ML.

Infrastructura locală completă — incluzând broker-ul Mosquitto MQTT configurat cu TLS, baza de date TimescaleDB cu hypertable-uri optimizate pentru serii temporale și

configurarea rețelei locale — constituie în ansamblu o soluție integrată care demonstrează fezabilitatea paradigmei local-first pentru aplicații IoT de producție.

1.4 Structura Lucrării

Lucrarea este organizată în șase capitole principale, fiecare abordând un aspect distinct al proiectului, de la fundamentele teoretice la detaliile de implementare și rezultatele experimentale.

Capitolul 2 — Arhitectura Sistemului prezintă viziunea arhitecturală de ansamblu a proiectului, detaliind principiile local-first și privacy-by-design care au ghidat toate deciziile de proiectare. Sunt descrise în detaliu cele cinci straturi ale arhitecturii, împreună cu fluxurile de date și justificările pentru alegerea fiecărei tehnologii componente. De asemenea, sunt analizate considerentele de securitate ale sistemului.

Capitolul 3 — Implementarea Hardware acoperă în detaliu componentele fizice ale sistemului: microcontrolerele ESP32 WROOM-32 și ESP32-CAM, senzorii utilizați (DHT22, MQ-2, PIR, BH1750) și actuatorii (releu, servo motor, buzzer). Sunt prezentate schemele de conexiuni, considerentele de alimentare electrică și configurarea rețelei fizice de noduri senzoriale.

Capitolul 4 — Implementarea Software descrie în detaliu toate componentele software ale sistemului: firmware-ul MicroPython pentru ESP32, configurarea broker-ului Mosquitto și a bazei de date TimescaleDB pe Raspberry Pi, arhitectura backend-ului NestJS și implementarea aplicației mobile React Native/Expo.

Capitolul 5 — Detecția de Anomalii prin Machine Learning este dedicat componentei de inteligență artificială a sistemului. Sunt prezentate fundamentele teoretice ale algoritmului Isolation Forest, procesul de colectare și preprocesare a datelor de antrenare, rezultatele antrenării și evaluarea performanțelor algoritmului pe date reale colectate de sistemul implementat.

Capitolul 6 — Testare și Evaluare prezintă metodologia de testare aplicată sistemului, rezultatele testelor funcționale și de performanță, analiza consumului de resurse pe Raspberry Pi și o evaluare comparativă față de soluțiile comerciale existente. Sunt discutate limitările sistemului și direcțiile de dezvoltare ulterioară.

Capitolul 2

Arhitectura Sistemului

2.1 Viziunea Arhitecturală

Proiectarea oricărui sistem software complex pornește de la un set de principii fundamentale care ghidează toate deciziile tehnice ulterioare. În cazul sistemului descris în prezenta lucrare, trei principii arhitecturale au constituit pilonii conceptuali ai întregului demers: localitatea datelor (*local-first*), confidențialitatea prin proiectare (*privacy by design*) și inteligența la marginea rețelei (*edge intelligence*). Înțelegerea acestor principii este esențială pentru a aprecia rațiunea din spatele fiecărei decizii tehnologice prezentate în capitolele ulterioare.

2.1.1 Principiul Local-First

Conceptul de *local-first software*, formalizat în literatura academică de Kleppmann et al. [1], descrie o paradigmă de proiectare în care datele sunt stocate și procesate în primul rând pe dispozitivele utilizatorului, nu pe servere externe. Aplicat în contextul IoT și al sistemelor smart home, principiul local-first presupune că toate funcționalitățile esențiale ale sistemului — colectarea datelor, stocarea, procesarea și controlul actuatorilor — trebuie să funcționeze în mod complet și corect în absența oricărei conexiuni la internet.

Din punct de vedere tehnic, implementarea principiului local-first în sistemul de față s-a materializat prin mai multe decizii de proiectare concrete. Broker-ul MQTT Mosquitto rulează pe Raspberry Pi 3B+, eliminând orice intermediar cloud în fluxul de comunicație dintre nodurile senzoriale și backend. Baza de date TimescaleDB este găzduită local, pe același dispozitiv gateway, asigurând că întregul istoric al datelor rămâne sub controlul exclusiv al utilizatorului. Backend-ul NestJS și algoritmul de machine learning rulează de asemenea pe Raspberry Pi, iar aplicația mobilă comunică direct cu adresa IP locală a gateway-ului, fără a tranzita niciun server extern.

Această abordare prezintă avantaje măsurabile față de modelul cloud-centric: latența end-to-end pentru o alertă generată de un senzor până la notificarea pe telefon se reduce la câteva zeci de milisecunde pe rețeaua locală, față de sute de milisecunde sau chiar secunde specifice rutelor prin cloud. Disponibilitatea sistemului devine independentă de furnizori externi sau de calitatea conexiunii la internet, aspect cu implicații practice semnificative în scenarii reale.

2.1.2 Confidențialitatea prin Proiectare

Privacy by design este un principiu formulat de Ann Cavoukian în anii '90 [3], ulterior adoptat ca cerință explicită de GDPR (Art. 25), care stipulează că protecția datelor personale trebuie integrată în arhitectura sistemelor informatice încă din faza de proiectare, nu adăugată

post-factum ca strat separat. Aplicat sistemului smart home, acest principiu are implicații directe și concrete.

Datele colectate de senzorii ambientali — temperaturi, mișcări, niveluri de gaze, imagini video — constituie date personale sensibile, deoarece permit inferirea comportamentului, rutinelor și prezenței persoanelor în locuință. În arhitectura propusă, aceste date nu traversează niciodată rețele publice și nu sunt accesibile unor terțe părți. Toate comunicațiile dintre nodurile ESP32 și gateway sunt protejate prin MQTT over TLS (portul 8883), prevenind interceptarea în cadrul rețelei locale. Autentificarea la nivelul API-ului NestJS se realizează prin token-uri JWT cu durată limitată, împiedicând accesul neautorizat. Aplicația mobilă nu include niciun SDK de analytics, advertising sau telemetrie terță parte.

2.1.3 Inteligența la Marginea Rețelei față de Inteligența în Cloud

Dihotomia *edge intelligence* versus *cloud intelligence* reprezintă una dintre dezbaterile centrale ale IoT-ului modern. Abordarea tradițională constă în transmiterea datelor brute de la senzori către platforme cloud (AWS IoT, Google Cloud IoT, Azure IoT Hub) unde sunt procesate de modele ML cu resurse computaționale nelimitate. Abordarea *edge* presupune rularea algoritmilor de ML pe dispozitive cu resurse limitate, amplasate aproape de sursa datelor.

Compromisul fundamental constă în faptul că modelele cloud beneficiază de putere de calcul practic nelimitată, permițând utilizarea unor arhitecturi complexe (rețele neuronale profunde, modele de mari dimensiuni), în timp ce modelele *edge* sunt constrânse de resursele hardware disponibile, dar beneficiază de latență redusă, funcționare offline și confidențialitate. Pentru cazul de utilizare specific al sistemului de față — detecția anomaliilor comportamentale pe date multidimensionale de la 4–5 senzori — algoritmul Isolation Forest reprezintă un compromis optim: are complexitate computațională moderată, adecvată pentru Raspberry Pi 3B+, și oferă performanțe excelente pe probleme de detecție a anomaliilor în date cu distribuție necunoscută, fără a necesita date etichetate pentru antrenare.

2.2 Arhitectura pe Straturi

Sistemul este structurat pe cinci straturi funcționale distincte, fiecare cu responsabilități bine delimitate și interfețe clar definite față de straturile adiacente. Această organizare verticală respectă principiul separării preocupărilor (*separation of concerns*) și facilitează mentenanța, testarea și evoluția independentă a fiecărei componente.

Figura 2.1 ilustrează schema arhitecturii pe straturi a sistemului.

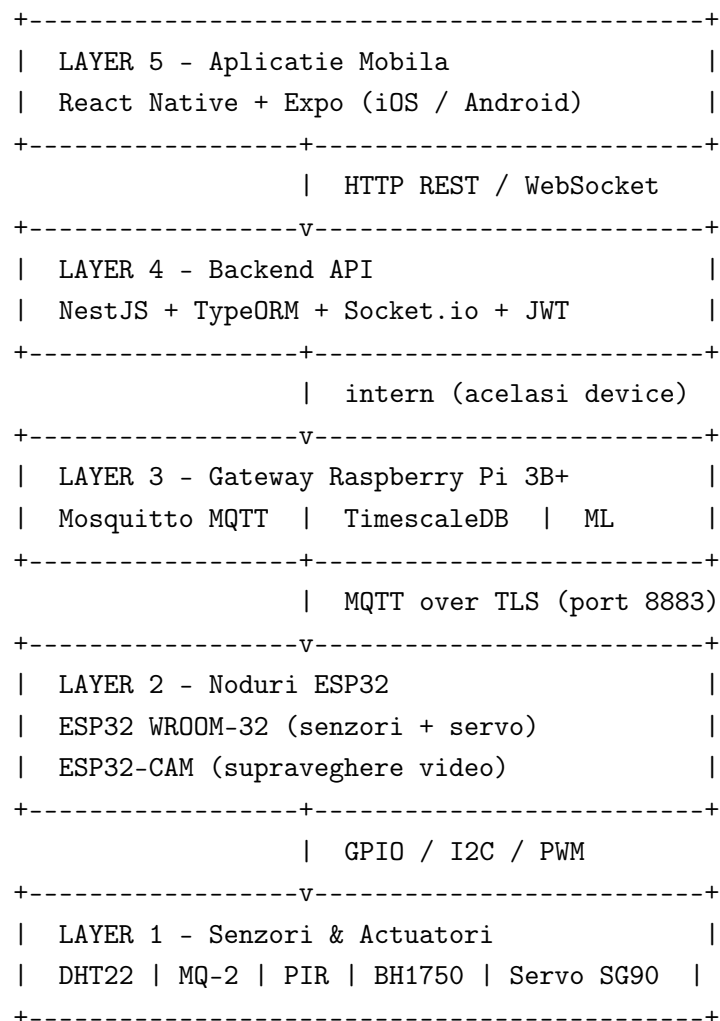


Figura 2.1. Arhitectura pe cinci straturi a sistemului Smart Home Local-First

2.2.1 Stratul 1 — Senzori și Actuatori

Primul strat al arhitecturii cuprinde totalitatea dispozitivelor fizice de intrare și ieșire conectate la nodurile microcontroler. Din perspectiva senzorilor, sistemul integrează patru tipuri distincte de traductoare, acoperind principalele dimensiuni de monitorizare ale unui mediu rezidențial.

Senzorul DHT22 asigură măsurarea temperaturii în intervalul $-40^{\circ}\text{C} \dots +80^{\circ}\text{C}$ cu o precizie de $\pm 0,5^{\circ}\text{C}$ și a umidității relative în intervalul $0\% \dots 100\%$ cu o precizie de $\pm 2-5\%$. Comunicația se realizează prin protocolul 1-Wire proprietar pe un singur pin digital. Senzorul MQ-2 este un detector de gaze combustibile și fum, bazat pe un element de oxid metalic semiconductor (SnO_2) al cărui rezistivitate se modifică în prezența gazelor reducătoare (metan, propan, hidrogen, monoxid de carbon, fum). Semnalul de ieșire este analogic, necesitând conversie analog-digitală prin ADC-ul intern al ESP32. Senzorul PIR (Passive Infrared) HC-SR501 detectează mișcarea prin variațiile de radiație infraroșie generate de deplasarea corpurilor calde în câmpul său de vedere (unghi de 120° , rază de 7 m), furnizând

un semnal digital binar. Senzorul BH1750 măsoară luminanța ambientală în intervalul 1–65535 lux prin comunicație I²C, oferind date cantitative pentru automatizarea iluminatului.

Din perspectiva actuatorilor, sistemul include un releu de putere pentru controlul echipamentelor electrice de curent alternativ (până la 10A/250VAC), un servo motor pentru aplicații de control mecanic (închidere/deschidere perdele), și un buzzer piezoelectric pentru alertare acustică locală.

2.2.2 Stratul 2 — Noduri ESP32

Al doilea strat este constituit din microcontrolerile ESP32, care joacă rolul de interfețe inteligente între lumea fizică a senzorilor și rețeaua locală. Modulul ESP32 WROOM-32, produs de Espressif Systems, integrează un procesor dual-core Xtensa LX6 la 240 MHz, 4 MB de memorie flash, 520 KB SRAM, conectivitate Wi-Fi 802.11 b/g/n și Bluetooth 4.2/BLE. Aceste specificații tehnice îl poziționează semnificativ deasupra microcontrolerelor clasice de tip Arduino, oferind resurse computaționale suficiente pentru rularea unui stivă de protocoale complete (TCP/IP, TLS, MQTT) și pentru preprocesarea datelor la nivel local.

Firmware-ul rulat pe nodurile ESP32 WROOM-32 este responsabil pentru: inițializarea și calibrarea senzorilor la pornire, citirea periodică a valorilor (la interval configurabil, implicit 30 de secunde), validarea și filtrarea datelor aberante, serializarea valorilor în format JSON și publicarea acestora pe topicurile MQTT corespunzătoare, precum și pentru abonarea la topicurile de comandă și executarea acțiunilor solicitate (activare releu, poziționare servo).

Nodul ESP32-CAM reprezintă o variantă specializată a familiei ESP32, echipată suplimentar cu un modul cameră OV2640 (2 MP, până la 1600×1200 pixeli) și un slot microSD pentru stocarea locală a imaginilor. Acesta operează independent față de nodul senzorial principal, gestionând captarea de imagini la cerere sau la detectarea mișcării, comprimarea JPEG și transmiterea unui flux video continuu MJPEG către aplicația mobilă.

2.2.3 Stratul 3 — Gateway Raspberry Pi 3B+

Raspberry Pi 3B+ constituie nucleul arhitectural al întregului sistem, concentrând patru subsisteme funcționale critice pe un singur dispozitiv cu consum energetic redus (5V/2,5A, aproximativ 5–7W în funcționare normală).

Broker-ul MQTT Mosquitto gestionează întreaga comunicație publish-subscribe între nodurile ESP32 și backend-ul NestJS. Mosquitto este un broker open-source de înaltă performanță, implementând standardul MQTT 3.1.1 și 5.0, capabil să gestioneze mii de conexiuni simultane pe hardware modest. Configurarea cu TLS mutual asigură criptarea end-to-end a tuturor mesajelor IoT. Topicurile MQTT sunt organizate ierarhic după schema `home/{room}/{sensor_type}` pentru date și `home/{room}/cmd/{actuator}` pentru comenzi.

TimescaleDB este extensia PostgreSQL optimizată pentru date de tip serie temporală, selectată pentru stocarea istoricului complet al citirilor senzorilor. TimescaleDB introduce conceptul de *hypertable* — o tabelă particionată automat pe dimensiunea temporală — care menține performanțe constante de inserare și interogare chiar și pe seturi de date cu milioane

de înregistrări. Funcțiile de *continuous aggregation* permit calculul eficient al statisticilor agregate (medii orare, zilnice, săptămânale) fără a reciti datele brute.

ML Engine-ul reprezintă componenta de inteligență artificială a sistemului, implementată ca un serviciu Python care rulează Isolation Forest din biblioteca `scikit-learn`. Serviciul consumă datele stocate în TimescaleDB pentru antrenare, evaluează periodic normalitatea citirilor curente și publică rezultatele evaluării (scor de anomalie, clasificare normal/anormal) pe topicul MQTT dedicat alertelor.

2.2.4 Stratul 4 — Backend API NestJS

Backend-ul NestJS reprezintă stratul de orchestrare și abstractizare al sistemului, expunând interfețe standardizate consumabile de aplicația mobilă. Ales pentru arhitectura sa modulară și orientată pe decoratori, NestJS organizează funcționalitățile în module independente: modulul de autentificare (JWT, `bcrypt` pentru hashing parole), modulul de senzori (CRUD pentru configurații și citiri), modulul de actuatori (comenzi și stare), modulul de alerte (stocare, marcarea ca citit, filtrare) și gateway-ul WebSocket (Socket.io, broadcast selectiv per client autentificat).

Comunicarea cu broker-ul MQTT se realizează intern, în cadrul aceluiași dispozitiv Raspberry Pi, prin biblioteca `mqtt.js`, eliminând latența de rețea pentru acest flux critic. Comunicarea cu TimescaleDB se realizează prin TypeORM, cu entități mapate pe structura hypertable-urilor.

2.2.5 Stratul 5 — Aplicația Mobilă React Native/Expo

Stratul de prezentare al sistemului este reprezentat de o aplicație mobilă cross-platform, dezvoltată cu React Native și Expo, compatibilă cu platformele iOS și Android. Aplicația adoptă o arhitectură bazată pe Zustand pentru managementul stării globale și React Navigation pentru navigarea între ecrane. Comunicarea cu backend-ul utilizează Axios pentru cererile HTTP REST și Socket.io-client pentru conexiunea WebSocket persistentă, prin care sunt primite actualizările în timp real ale datelor senzorilor și notificările de alertă. Interfața vizuală adoptă o temă întunecată caldă (*warm dark*), construită pe paleta de culori stone și amber, optimizată pentru utilizare în condiții de luminozitate redusă.

2.3 Fluxul de Date (End-to-End)

Înțelegerea completă a arhitecturii necesită urmărirea datelor prin toate straturile sistemului, atât pe calea de monitorizare (de la senzori la ecranul utilizatorului), cât și pe calea de comandă (de la interfața mobilă la actuatori) și pe calea de alertare (de la detectarea anomaliei la notificarea utilizatorului).

2.3.1 Fluxul de Monitorizare (Senzori → Mobil)

Ciclul de monitorizare începe cu citirea periodică a valorilor senzorilor de către firmware-ul ESP32, la intervalele configurate. Valorile sunt validate local (eliminarea citirilor

ieșite din domeniu sau a erorilor de comunicație), serializate în format JSON și publicate pe topicul MQTT corespunzător (de exemplu, `home/living/temperature`). Broker-ul Mosquitto recepționează mesajul și îl livrează backend-ului NestJS, care persistează înregistrarea în TimescaleDB, actualizează cache-ul intern cu ultima valoare și emite un eveniment WebSocket pe canalul `sensor-update` către toți clienții mobili autentificați. Întregul flux se desfășoară, în condiții normale de rețea locală, în mai puțin de 100 de milisecunde de la momentul citirii senzorului.

2.3.2 Fluxul de Comandă (Mobil → Actuator)

Calea de comandă funcționează în sens invers față de fluxul de monitorizare. Utilizatorul interacționează cu un control din aplicația mobilă (de exemplu, un comutator pentru releul de putere). Aplicația emite o cerere HTTP POST către endpoint-ul REST corespunzător al backend-ului NestJS, cu payload-ul comenzii și token-ul JWT de autentificare în header-ul Authorization. Backend-ul validează token-ul, persistă comanda și publică un mesaj MQTT pe topicul de comandă al actuatorului (`home/living/cmd/relay`). Nodul ESP32, abonat la acest topic, recepționează comanda și execută acțiunea fizică corespunzătoare. Starea actuatorului este publicată înapoi pe un topic de confirmare, închizând bucla de feedback.

2.3.3 Fluxul de Alertare (Anomalie ML → Notificare)

Calea de alertare este declanșată de ML Engine-ul care rulează periodic pe Raspberry Pi. La fiecare ciclu de evaluare, serviciul Python extrage ultimele citiri ale senzorilor din TimescaleDB și le supune scoringului Isolation Forest. Dacă scorul de anomalie depășește pragul calibrat, ML Engine-ul publică un mesaj MQTT pe topicul `home/alerts/anomaly`, cu detalii despre tipul anomaliei, valorile implicate și scorul de încredere. Backend-ul NestJS persistă alerta și emite imediat un eveniment WebSocket de tipul `alert-new` către toți clienții mobili conectați. Aplicația mobilă afișează o notificare locală și actualizează ecranul de alerte cu noua înregistrare.

2.4 Alegerea Tehnologiilor

Fiecare decizie tehnologică din cadrul proiectului a fost fundamentată pe o analiză comparativă a alternativelor disponibile, luând în considerare criteriile specifice ale unui sistem local-first pentru IoT rezidențial.

2.4.1 NestJS față de alternativele backend

Alternativele evaluate pentru stratul de backend au inclus Flask și Django (Python), Express.js (Node.js minimal) și Spring Boot (Java). NestJS a fost ales din mai multe motive convergente. Arhitectura sa modulară și sistemul de injecție de dependențe reduc semnificativ complexitatea accidentală a proiectului comparativ cu Express.js. Utilizarea TypeScript ca limbaj nativ oferă siguranță de tip (*type safety*) end-to-end, reducând clasa de erori runtime și facilitând refactorizarea. Suportul nativ pentru WebSocket prin decoratori și gateway-uri

Socket.io elimină necesitatea configurării manuale a unui server WebSocket separat. Față de soluțiile Python (Django/Flask), NestJS oferă performanțe superioare la gestionarea conexiunilor WebSocket concurente, aspect critic pentru un sistem cu actualizări frecvente în timp real.

2.4.2 React Native față de Flutter

Principalele alternative evaluate pentru aplicația mobilă au fost Flutter (Google, Dart) și React Native (Meta, JavaScript/TypeScript). Alegerea React Native/Expo s-a bazat pe: ecosistemul JavaScript extrem de matur și bogat în librării de componentă, alinierea cu backend-ul TypeScript pentru partajarea de tipuri și logică de validare, instrumentul Expo Go care permite iterarea rapidă fără build nativ la fiecare modificare, și compatibilitatea bibliotecilor de grafice cu cerințele de vizualizare ale proiectului. Flutter ar fi oferit performanțe de randare superioare prin compilarea nativă, însă costul învățării limbajului Dart și ecosistemul mai puțin matur pentru librăriile IoT-specifice au constituit dezavantaje decisive pentru contextul acestui proiect.

2.4.3 MQTT față de HTTP polling

Alegerea MQTT ca protocol de comunicație IoT a fost facilă față de alternativa HTTP polling. Modelul publish-subscribe al MQTT este natural adaptat pentru scenariile IoT unde mulți producători de date (noduri senzoriale) comunică asincron cu mulți consumatori (backend, aplicații de monitorizare). Latența MQTT este semnificativ mai mică față de HTTP polling, deoarece mesajele sunt livrate imediat la publicare. Consumul de bandă și de energie este minim, dat fiind că headerele MQTT sunt de ordinul a zeci de bytes. Nivelurile de QoS (Quality of Service) oferite de protocol — QoS 0 (cel mult o dată), QoS 1 (cel puțin o dată), QoS 2 (exact o dată) — permit calibrarea compromisului între fiabilitate și overhead în funcție de tipul datelor.

2.4.4 TimescaleDB față de PostgreSQL simplu

TimescaleDB extinde PostgreSQL cu optimizări specifice datelor de tip serie temporală fără a sacrifica compatibilitatea cu ecosistemul SQL standard. Față de un PostgreSQL simplu, TimescaleDB oferă: partiționarea automată a datelor pe dimensiunea temporală prin hypertable-uri, cu menținerea performanțelor de inserare și interogare la scară; funcții de agregate continue (*continuous aggregations*) care calculează și mențin automat statistici pre-agregate pentru interogări rapide pe intervale de timp; politici de compresie automată a datelor istorice cu rapoarte de compresie de 10–20× față de tabelele necomprimate. Față de soluții dedicate de time-series cum ar fi InfluxDB sau Prometheus, TimescaleDB oferă avantajul compatibilității complete SQL și al utilizării ecosistemului PostgreSQL (TypeORM, pgAdmin), reducând complexitatea infrastructurii.

2.4.5 Isolation Forest față de reguli cu praguri fixe

Sistemele clasice de alertare în smart home utilizează reguli cu praguri fixe: dacă temperatura depășește 35°C , se generează alertă; dacă concentrația MQ-2 depășește un prag absolut, se generează alertă. Această abordare este simplă și transparentă, dar prezintă limitări fundamentale: pragurile optime variază între locuințe, anotimpuri și obiceiuri ale utilizatorului; nu captează corelațiile dintre multiple dimensiuni (de exemplu, o temperatură de 30°C este normală în august dar anormală în ianuarie); și tind să genereze rate ridicate de fals-pozitive sau fals-negative dacă sunt calibrate incorect.

Algoritmul Isolation Forest, propus de Liu et al. [2], abordează diferit problema detecției anomaliilor: în loc să definească normalul și să identifice devierile de la el, izolează explicit punctele anormale prin partiționarea aleatorie recursivă a spațiului de date. Punctele anormale, fiind rare și diferite de masa datelor normale, sunt izolate cu un număr mai mic de partiționări, obținând un scor de anomalie ridicat. Avantajele pentru cazul de față sunt: algoritmul nu necesită date etichetate pentru antrenare (învățare nesupervizată); are complexitate computațională $O(n \log n)$ și consum de memorie $O(n)$, adecvat pentru Raspberry Pi; se adaptează automat la tiparele specifice ale utilizatorului pe măsura acumulării datelor de antrenare.

2.5 Considerații de Securitate

Securitatea unui sistem smart home este o preocupare de prim ordin, dat fiind că dispozitivele controlate (releu de putere, actuatori mecanici) pot influența direct siguranța fizică a locuinței. Deși sistemul propus operează exclusiv pe rețeaua locală — eliminând o categorie întreagă de vectori de atac specifici serviciilor cloud — există totuși amenințări relevante care trebuie adresate prin mecanisme tehnice adecvate.

Autentificarea și autorizarea la nivelul API-ului NestJS se realizează prin JSON Web Tokens (JWT) cu o durată de expirare de șapte zile. Token-urile sunt semnate cu un secret configurat ca variabilă de mediu, prevenind falsificarea. Parola utilizatorului este stocată exclusiv sub formă de hash bcrypt (factor de cost 12), astfel încât chiar și în cazul unui acces neautorizat la baza de date, parola în clar nu poate fi recuperată în timp util. Toate endpoint-urile API, cu excepția rutei de login, sunt protejate prin middleware-ul de autentificare JWT.

Criptarea comunicațiilor MQTT este asigurată prin TLS (Transport Layer Security) pe portul 8883. Certificatele TLS sunt generate local și stocate pe Raspberry Pi, iar nodurile ESP32 sunt configurate cu certificatul CA corespunzător. Această măsură previne interceptarea sau modificarea mesajelor MQTT de către un potențial atacator cu acces la rețeaua Wi-Fi locală.

Izolarea rețelei constituie primul și cel mai important strat de securitate al sistemului. Prin design, backend-ul NestJS expune portul API exclusiv pe interfața de rețea locală (LAN), fără port-forwarding sau expunere directă pe internet. Utilizatorul care dorește acces de la

distanță trebuie să configureze explicit o soluție de VPN (recomandat WireGuard), menținând controlul complet al suprafeței de atac.

Sistemul nu implementează autentificare la nivelul MQTT (autentificare per-client cu username/parolă), aspect identificat ca limitare actuală și direcție de îmbunătățire pentru versiunile viitoare. De asemenea, posibilitatea accesului de la distanță prin VPN, deși menționată, nu face obiectul implementării prezente, focalizată pe scenariul de utilizare locală.

Capitolul 3

Implementarea Hardware

3.1 Prezentare Generală

Arhitectura fizică a sistemului smart home descris în această lucrare este organizată pe două niveluri ierarhice distincte: nivelul de percepție și acțiune, reprezentat de nodurile embedded bazate pe microcontrolere ESP32, și nivelul de procesare și orchestrare, asigurat de un calculator monoplacă Raspberry Pi 3B+.

La nivelul de percepție funcționează doi noduri independente. Primul nod — construit pe microcontroler ESP32 WROOM-32 — are rolul de a colecta date de la senzori de mediu și de a controla actuatorii din camera de zi. Al doilea nod — realizat pe modulul ESP32-CAM AI-Thinker — asigură supravegherea vizuală a intrării și holului, furnizând un flux video continuu accesibil din aplicația mobilă.

La nivelul de procesare, Raspberry Pi 3B+ îndeplinește rolul de gateway local și server de aplicații. Acesta găzduiește broker-ul MQTT, serviciul NestJS cu API REST și WebSocket, baza de date PostgreSQL și motorul de detecție a anomaliilor bazat pe Isolation Forest. Toate comunicațiile între noduri și server se desfășoară exclusiv în rețeaua locală Wi-Fi, în conformitate cu principiul local-first enunțat în capitolul anterior.

Prototipul prezentat în această lucrare este realizat pe breadboard, configurație adecvată fazei de cercetare și validare funcțională. O eventuală trecere în producție ar implica proiectarea și fabricarea unui PCB dedicat, aspect care depășește sfera prezentei lucrări.

3.2 Microcontrolere Utilizate

3.2.1 ESP32 WROOM-32 — Nodul de Senzori

ESP32 WROOM-32 este un modul de tip System-on-Module (SoM) produs de Espressif Systems, proiectat pentru aplicații IoT care necesită conectivitate wireless și putere de calcul ridicată la un cost redus. Nucleul modulului este cipul ESP32-D0WDQ6, cu arhitectură dual-core Xtensa LX6, frecvență maximă de operare de 240 MHz și memorie flash internă de 4 MB. Memoria SRAM disponibilă este de 520 KB, suficientă pentru stocarea programului firmware și a structurilor de date asociate senzorilor.

Din punct de vedere al conectivității wireless, modulul integrează Wi-Fi 802.11 b/g/n (2,4 GHz) și Bluetooth 4.2 (clasic și BLE), ambele pe o antenă PCB integrată. Pentru această aplicație se utilizează exclusiv interfața Wi-Fi, prin care nodul publică datele de telemetrie către broker-ul MQTT de pe Raspberry Pi.

Perifericele disponibile sunt deosebit de versatile: 30 de pini GPIO configurabili, un convertor analog-digital (ADC) de 12 biți cu 18 canale, interfețe I²C, SPI, UART și capacitate

de generare PWM pe aproape orice pin. Tensiunea de operare este de 3,3 V, însă unii pini tolerează semnale de 5 V ca intrare.

Alegerea acestui microcontroler a fost motivată de mai mulți factori. Documentația extensivă și comunitatea vastă de utilizatori reduc semnificativ efortul de depanare. ADC-ul de 12 biți este necesar pentru citirea valorilor analogice de la senzorul MQ-2 de gaz. Interfața I²C este utilizată pentru comunicarea cu senzorul BH1750 de lumină. Capacitatea PWM pe GPIO18 permite controlul precis al servomotorului SG90. Nu în ultimul rând, prețul accesibil al modulului (aproximativ 5–7 EUR) contribuie la menținerea costului total al sistemului la un nivel rezonabil.

3.2.2 ESP32-CAM AI-Thinker — Nodul Cameră

Modulul ESP32-CAM AI-Thinker este o soluție integrată care combină un microcontroler ESP32-S cu un modul de cameră OV2640 de 2 megapixeli, destinată aplicațiilor de supraveghere video cu cost redus. Pe lângă procesorul ESP32-S, modulul dispune de o memorie PSRAM (Pseudo-Static RAM) de 4 MB, necesară pentru stocarea temporară a cadrelor video înainte de transmisie, și un slot microSD pentru înregistrare locală opțională.

Senzorul de imagine OV2640 suportă rezoluții de până la UXGA (1600×1200 pixeli) și poate transmite fluxuri video compresate în format MJPEG. Pentru aplicația prezentată, rezoluția utilizată în mod curent este VGA (640×480), care oferă un echilibru acceptabil între calitatea imaginii și lățimea de bandă consumată în rețeaua locală. Modulul dispune și de un LED flash pe GPIO4, utilizabil pentru iluminarea scenei în condiții de lumină redusă.

O particularitate importantă a acestui modul este absența unui convertor USB-UART integrat pe placă. În consecință, programarea firmware-ului necesită utilizarea unui adaptor extern FTDI (sau similar), procedură detaliată în secțiunea 3.7.

Motivele alegerii ESP32-CAM sunt în principal economice și funcționale: modulul oferă integrare completă cameră + Wi-Fi + microcontroler la un preț de aproximativ 8 EUR, biblioteca de streaming MJPEG este disponibilă nativ în framework-ul Arduino pentru ESP32, iar dimensiunea compactă permite montarea discretă la intrarea locuinței.

3.3 Senzori Utilizați

3.3.1 DHT22 — Temperatură și Umiditate

Senzorul DHT22 (cunoscut și sub denumirea AM2302) este un senzor digital de temperatură și umiditate relativă. Domeniul de măsurare a temperaturii este cuprins între -40°C și $+80^{\circ}\text{C}$, cu o precizie de $\pm 0,5^{\circ}\text{C}$, iar umiditatea relativă poate fi citită în intervalul 0–100% cu o precizie de $\pm 2\text{--}5\%$.

Comunicarea cu microcontrolerul se realizează printr-un protocol proprietar one-wire pe un singur pin de date. Linia DATA necesită o rezistență pull-up de 10 k Ω la tensiunea de alimentare (3,3 V) pentru funcționare corectă. Intervalul minim între două citiri succesive este de 2 secunde, limitat de procesul intern de conversie a senzorului. Senzorul este conectat la GPIO4 al ESP32 WROOM-32 și alimentat de la pinul 3,3 V al microcontrolerului.

3.3.2 MQ-2 — Gaz și Fum

Senzorul MQ-2 este un detector electrochimic capabil să detecteze o gamă largă de gaze inflamabile și vapori: GLP (gaz lichefiat petrolier), butan, propan, metan, hidrogen, fum și monoxid de carbon (CO). Modulul furnizează atât un semnal analogic (0–3,3 V, corespunzând unui interval ADC de 0–1023 unități) cât și un semnal digital de prag, reglabil prin potențiometrul integrat pe modul.

Senzorul necesită alimentare la 5 V (curentul de consum al elementului sensibil depășind capacitatea regulatorului de 3,3 V al ESP32) și un timp de preîncălzire de aproximativ 20 de secunde după pornire, înainte ca valorile citite să fie stabile și reprezentative. Pragul de alertă utilizat în firmware este de 350 de unități ADC, valoare determinată empiric în condiții normale de ambient. Semnalul analogic este citit de pe GPIO34 (canalul ADC1_CH6), iar semnalul digital de prag de pe GPIO35 — ambii pini configurați exclusiv ca intrări.

3.3.3 HC-SR501 — Mișcare PIR

Senzorul de mișcare HC-SR501 funcționează pe principiul detecției infraroșului pasiv (PIR — *Passive InfraRed*). Elementul sensibil detectează variațiile de radiație infraroșie termică emise de corpuri umane în mișcare, fără a emite nicio radiație proprie. Raza de detecție este de până la 7 metri, cu un unghi de acoperire de 120°.

Modulul se alimentează la 5 V și furnizează un semnal digital de ieșire de 3,3 V (compatibil direct cu intrările ESP32), activ HIGH pe durata detecției de mișcare. Sensibilitatea detectivă și durata menținerii semnalului HIGH după detecție sunt reglabile prin intermediul a doi potențiometre aflați pe spatele modulului. Senzorul este conectat la GPIO27 al ESP32.

3.3.4 BH1750 — Lumină Ambientală

BH1750 este un senzor digital de iluminanță (luminos ambientală) cu rezoluție de 16 biți, capabil să măsoare niveluri de lumină în intervalul 1–65535 lux. Comunicarea cu microcontrolerul se realizează prin interfața I²C, la adresa 0x23 atunci când pinul ADDR este conectat la GND (adresa alternativă 0x5C se obține prin conectarea ADDR la VCC).

Senzorul se alimentează la 3,3 V, ceea ce îl face compatibil direct cu pinii ESP32. Pinii I²C utilizați sunt GPIO21 (SDA) și GPIO22 (SCL), care reprezintă pinii I²C implicați ai ESP32. Datele de iluminanță sunt utilizate atât pentru logica de automatizare a iluminatului (control releu și servo perdele) cât și ca *feature* în algoritmul de detecție a anomaliilor comportamentale.

3.4 Actuatori Utilizați

3.4.1 Modulul Releu 4 Canale

Modulul de releu cu 4 canale este un circuit de comandă care permite controlul sarcinilor electrice de tensiune și curent ridicate (230 VAC, 50 Hz în rețeaua electrică de uz casnic) prin intermediul semnalelor logice de joasă tensiune furnizate de microcontroler. Fiecare canal este izolat optic prin optocupluri, protejând astfel microcontrolerul de eventualele

perturbații electrice generate de sarcinile comutate.

Releele sunt de tip activ LOW: starea activată (contact normal-deschis închis) corespunde unui semnal LOW (0 V) pe pinul de intrare. Acest comportament trebuie luat în considerare la inițializarea firmware-ului pentru a evita activarea accidentală a releelor la pornire. Bobina fiecărui releu necesită 5 V și aproximativ 70–80 mA, motiv pentru care modulul este alimentat dintr-o sursă externă și nu direct din pinii GPIO ale ESP32. Capacitatea maximă de comutare este de 10 A la 250 VAC sau 10 A la 30 VDC per canal. Pinii de control sunt GPIO26, GPIO25, GPIO33 și GPIO32.

3.4.2 Buzzer Activ

Buzzer-ul activ este un transductor electroacustic cu oscilator intern integrat, care produce un semnal sonor continuu atunci când i se aplică tensiune de alimentare (5 V), fără a necesita un semnal PWM extern. Simplitatea de control — activare/dezactivare prin nivel logic pe GPIO5 — îl face adecvat pentru alerte locale de urgență, în special în scenariile de detecție a gazului sau tentativă de efracție.

3.4.3 Servomotorul SG90 — Control Perdele

SG90 este un micro-servomotor cu angrenaje din plastic, utilizat pe scară largă în aplicații robotice și de automatizare la scară mică. Oferă un cuplu de 1,8 kg·cm la 5 V și poate efectua rotații în intervalul 0°–180°. Controlul poziției se realizează prin semnal PWM la frecvența de 50 Hz: lățimea impulsului variază de la 0,5 ms (corespunzând poziției de 0°) la 2,5 ms (corespunzând poziției de 180°).

În cadrul sistemului, servomotorul este utilizat pentru acționarea unui mecanism simplu de deschidere/închidere a perdelelor, cu trei poziții predefinite: 0° (perdele complet închise), 90° (perdele la jumătate) și 180° (perdele complet deschise). Aceasta permite automatizarea iluminatului natural în funcție de nivelul de lumină ambientală măsurat de senzorul BH1750.

Alimentarea servomotorului trebuie realizată obligatoriu dintr-o sursă externă de 5 V, și nu din pinii de alimentare ai ESP32. Un servomotor SG90 poate consuma curenți de vârf de 500–600 mA în condiții de sarcină, valoare ce depășește cu mult capacitatea de furnizare de curent a regulatorului intern al ESP32 (maxim 40 mA per pin GPIO, respectiv câteva sute de mA pe pinul 3V3). Semnalul PWM este generat pe GPIO18, pin cu capacitate PWM nativă.

3.5 Schema de Conexiuni

Tabelul 3.1 prezintă complet conexiunile dintre componentele hardware și pinii corespunzători ai ESP32 WROOM-32, împreună cu observațiile relevante pentru asamblarea corectă a prototipului pe breadboard.

Tabela 3.1. Conexiunile componentelor pe nodul ESP32 WROOM-32

Componentă	Pin componentă	Pin ESP32	Obs.
DHT22	VCC	3,3V (intern)	—
DHT22	GND	GND	—
DHT22	DATA	GPIO4	10 k Ω pull-up la 3,3V
MQ-2	VCC	5V (extern)	Nu din ESP32!
MQ-2	GND	GND	—
MQ-2	A0	GPIO34	ADC1_CH6
MQ-2	D0	GPIO35	Input-only
PIR HC-SR501	VCC	5V (extern)	—
PIR HC-SR501	GND	GND	—
PIR HC-SR501	OUT	GPIO27	—
BH1750	VCC	3,3V (intern)	—
BH1750	GND	GND	—
BH1750	SDA	GPIO21	I ² C default
BH1750	SCL	GPIO22	I ² C default
BH1750	ADDR	GND	Adresă 0x23
Relay 4ch	VCC	5V (extern)	—
Relay 4ch	GND	GND	—
Relay 4ch	IN1	GPIO26	Active LOW
Relay 4ch	IN2	GPIO25	Active LOW
Relay 4ch	IN3	GPIO33	Active LOW
Relay 4ch	IN4	GPIO32	Active LOW
Buzzer	+	GPIO5	—
Buzzer	—	GND	—
Servo SG90	VCC roșu	5V (extern!)	Nu din ESP32!
Servo SG90	GND maro	GND	—
Servo SG90	PWM galben	GPIO18	50 Hz PWM

3.6 Alimentarea Sistemului

Diversitatea componentelor și cerințele lor electrice diferite impun o strategie atentă de alimentare. Modulul de releu, senzorul MQ-2, senzorul PIR HC-SR501 și servomotorul SG90 necesită toți tensiunea de 5 V și pot solicita curenți cumulativi de ordinul amperilor în condiții de operare simultană. Aceste valori depășesc capacitatea de furnizare a regulatorului intern al ESP32 sau a unui port USB standard.

Soluția adoptată este utilizarea unei surse externe de 5 V cu curentul de ieșire de 3 A, conectată direct la șinele de alimentare ale breadboard-ului. Microcontrolerul ESP32 WROOM-32 este alimentat fie prin conectorul USB propriu, fie prin pinul VIN (care acceptă 5 V direct), extras din aceeași șină de 5 V a breadboard-ului. Regulatorul LDO integrat

pe modulul ESP32 generează apoi tensiunea internă de 3,3 V, disponibilă pe pinul 3V3 și utilizată pentru alimentarea senzorilor DHT22 și BH1750, care au consum redus (sub 5 mA fiecare).

Este esențial ca masa (GND) surselor de 5 V externe și masa ESP32 să fie conectate în comun pe breadboard, altfel referințele de tensiune diferă și semnalele logice devin nesigure. Principiul fundamental care trebuie respectat este că servo-ul și releele nu trebuie alimentate niciodată direct din pinii GPIO sau de pe pinul 3V3 al ESP32 — o astfel de conexiune riscă distrugerea permanentă a microcontrolerului prin supracurent.

3.7 Nodul Cameră ESP32-CAM — Programare și Configurare

Absența unui convertor USB-UART integrat pe modulul ESP32-CAM impune utilizarea unui programator extern pentru încărcarea firmware-ului. Procedura standard folosește un adaptor FTDI (FT232RL sau similar), care realizează conversia USB-Serial necesară comunicării cu portul UART0 al ESP32.

Conexiunile necesare între adaptorul FTDI și modulul ESP32-CAM sunt următoarele: GND (FTDI) → GND (ESP32-CAM), VCC 5V (FTDI) → 5V (ESP32-CAM), TX (FTDI) → U0RXD / GPIO3 (ESP32-CAM), RX (FTDI) → U0TXD / GPIO1 (ESP32-CAM).

Pentru intrarea în modul de programare (*flash mode*), pinul GPIO0 al ESP32-CAM trebuie conectat la GND înainte de aplicarea tensiunii de alimentare. Această configurare determină boot-loader-ul să aștepte transferul firmware-ului prin UART în loc să execute codul din flash. Pașii de urmat sunt: (1) se conectează GPIO0 la GND prin intermediul unui jumper sau fir pe breadboard; (2) se conectează adaptorul FTDI la calculator prin USB; (3) se apasă butonul RESET de pe modulul ESP32-CAM; (4) se efectuează upload-ul firmware-ului din Arduino IDE sau PlatformIO; (5) după finalizarea încărcării, se îndepărtează jumper-ul GPIO0-GND; (6) se apasă din nou butonul RESET pentru pornirea normală cu noul firmware.

Din punct de vedere funcțional, nodul cameră rulează un server MJPEG pe portul 80. Fluxul video continuu este accesibil la adresa [http://\[IP\]/stream](http://[IP]/stream), iar capturile individuale (snapshot) la [http://\[IP\]/capture](http://[IP]/capture). Spre deosebire de nodul de senzori, nodul cameră nu publică date prin MQTT și nu participă la logica de automatizare sau detecție a anomaliilor — el este un nod pasiv, accesat direct din aplicația mobilă atunci când utilizatorul dorește să vizualizeze imaginea live a intrării.

3.8 Considerații Practice pentru Breadboard

Asamblarea unui prototip funcțional pe breadboard implică o serie de constrângeri practice care, dacă nu sunt respectate, pot genera defecțiuni intermitente dificil de diagnosticat.

Convenția de culori pentru fire este recomandată a fi respectată în mod consistent pe tot parcursul asamblării: roșu pentru VCC (indiferent de tensiune — 3,3 V sau 5 V), negru pentru GND, iar culorile neutre (galben, albastru, verde, portocaliu) pentru liniile de semnal.

Această practică reduce semnificativ riscul inversării accidentale a polarității, mai ales în condițiile unui număr mare de conexiuni pe breadboard.

Senzorul MQ-2 necesită un timp de preîncălzire de aproximativ 20 de secunde după pornire înainte ca rezistența elementului sensibil să se stabilizeze și valorile ADC să fie reprezentative. Firmware-ul ignoră primele citiri după pornire pentru a evita alerte false generate de instabilitatea de start-up a senzorului.

Senzorul DHT22 trebuie poziționat fizic la distanță de componentele care generează căldură locală, în special bobinele releelor active. Un relee activat generează un câmp magnetic și disipă căldură care poate influența temperatura măsurată de DHT22, introducând erori sistematice în datele de telemetrie.

Servomotorul SG90 prezintă vibrații mecanice în timpul rotației care pot slăbi conexiunile pe breadboard. Se recomandă fixarea firelor servomotorului cu bandă adezivă sau cleme de fixare suplimentare pentru a preveni întreruperea intermitentă a contactelor.

Modulul ESP32-CAM se încălzește notabil în timpul transmisiei video continue, disipând câțiva wați prin carcasa cipului. Este recomandat să se asigure un spațiu liber în jurul modulului pentru convecție naturală a aerului, evitând acoperirea sau izolarea termică accidentală în timpul testelor de lungă durată.

Capitolul 4

Implementarea Software

Capitolul urmează să fie redactat. Va descrie: firmware-ul MicroPython pentru nodurile ESP32, configurarea Mosquitto MQTT și TimescaleDB pe Raspberry Pi, arhitectura backend-ului NestJS (module, controllere, servicii, gateway WebSocket) și implementarea aplicației mobile React Native/Expo (ecrane, navigare, store Zustand, integrare API).

Capitolul 5

Detectia Anomaliilor prin Machine Learning

Capitolul urmează să fie redactat. Va descrie: fundamente teoretice ale algoritmului Isolation Forest (Liu et al., 2008), schema de colectare și preprocesare a datelor de antrenare, implementarea Python cu scikit-learn pe Raspberry Pi, evaluarea performanțelor și calibrarea pragului de anomalie pe date reale.

Capitolul 6

Testare și Evaluare

Capitolul urmează să fie redactat. Va descrie: metodologia de testare (teste funcționale, teste de performanță, teste de regresie), rezultatele testelor, analiza consumului de resurse pe Raspberry Pi, comparație cu soluțiile comerciale și direcții de dezvoltare ulterioară.

Concluzii

Secțiunea urmează să fie completată după finalizarea tuturor capitolelor de implementare. Va sintetiza cele mai importante contribuții ale lucrării, va evalua gradul de îndeplinire a obiectivelor stabilite în Capitolul 1 și va propune direcții de cercetare și dezvoltare ulterioară.

Bibliografie

Bibliografie

- [1] Kleppmann, M., Wiggins, A., van Hardenberg, P., McGranaghan, M., *Local-First Software: You Own Your Data, in Spite of the Cloud*. ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019), pp. 154–178, 2019.
- [2] Liu, F. T., Ting, K. M., Zhou, Z.-H., *Isolation Forest*. IEEE International Conference on Data Mining (ICDM), pp. 413–422, 2008.
- [3] Cavoukian, A., *Privacy by Design: The 7 Foundational Principles — Implementation and Mapping of Fair Information Practices*. Information and Privacy Commissioner of Ontario, Canada, 2009.
- [4] Espressif Systems, *ESP32 Technical Reference Manual v5.1*. Disponibil la: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf (accesat: mai 2025).
- [5] *** *Smart Home — Worldwide. Market Data Report*. Statista, 2024. Disponibil la: <https://www.statista.com/outlook/dmo/smart-home/worldwide> (accesat: mai 2025).
- [6] *** *Eclipse Mosquitto: An open source MQTT broker*. Eclipse Foundation, 2024. Disponibil la: <https://mosquitto.org/> (accesat: mai 2025).
- [7] *** *TimescaleDB Documentation*. Timescale, Inc., 2024. Disponibil la: <https://docs.timescale.com/> (accesat: mai 2025).
- [8] *** *NestJS — A progressive Node.js framework*. Disponibil la: <https://nestjs.com/> (accesat: mai 2025).
- [9] *** *React Native — Learn once, write anywhere*. Meta Platforms, Inc., 2024. Disponibil la: <https://reactnative.dev/> (accesat: mai 2025).
- [10] *** *Expo Documentation*. Expo Technology, Inc., 2024. Disponibil la: <https://docs.expo.dev/> (accesat: mai 2025).

DECLARAȚIE DE AUTENTICITATE A LUCRĂRII DE FINALIZARE A STUDIILOR

Subsemnatul _____,
legitimăt cu _____ seria _____ nr. _____,
CNP _____,
autorul lucrării _____

elaborată în vederea susținerii examenului de finalizare a studiilor de _____
organizat de către Facultatea _____ din cadrul Universității Politehnica
Timișoara, sesiunea _____ a anului universitar _____, coordonator _____,
luând în considerare art. 34 din Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale,
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate,
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor,
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen.

Timișoara,

Data _____

Semnătura _____

Declarația se completează „de mână” și se inserează în lucrarea de finalizare a studiilor, la sfârșitul acesteia, ca parte integrantă.